



Module 5 – Déploiement et exécution des applications

Automatiser les déploiements de TrackR

Kubernetes – Orchestration des conteneurs

Objectifs du module

À la fin de ce module, vous saurez :

- déployer une application avec un **Deployment** et la mettre à l'échelle, à la main ou avec un **HPA** ;
- mettre à jour **sans interruption** et revenir en arrière ;
- protéger le trafic avec les **sondes** de santé ;
- exposer TrackR avec les **Services**, le DNS et l'Ingress ;
- automatiser avec **Kustomize**, **Helm** et le GitOps.

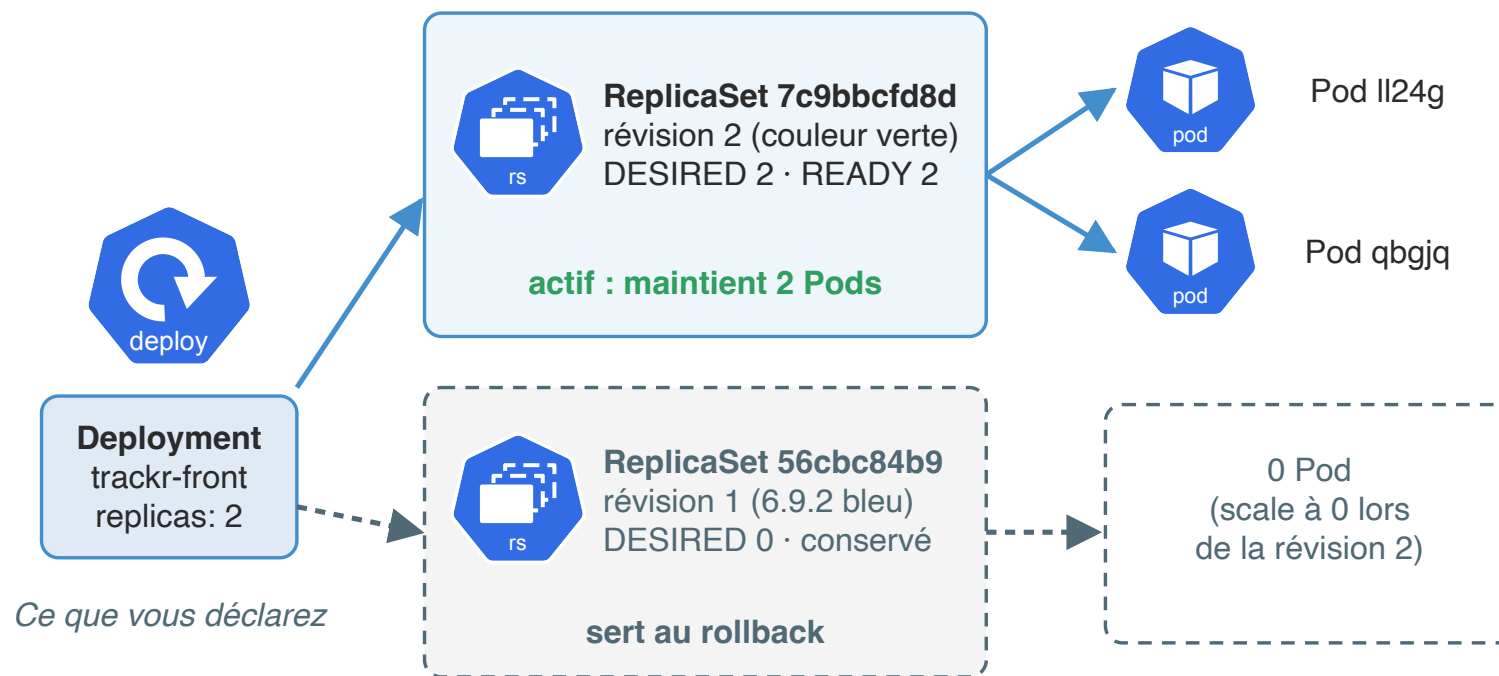


Plan du module

1. Deployment, ReplicaSet et mise à l'échelle (manuelle, HPA, autres contrôleurs).
 2. Mises à jour : RollingUpdate, Recreate, historique et rollback.
 3. Sondes de santé : startup, readiness, liveness.
 4. Exposition : Services, EndpointSlices, DNS interne, Ingress.
 5. Automatisation : déclaratif, Kustomize, Helm, CI/CD et GitOps.
- Durée : 2 h 15, dont deux démos et le TP 5 « Déployer TrackR complet ».

1. Deployment, ReplicaSet et mise à l'échelle

Du Deployment aux Pods



Chaque modification du modèle de Pod (pod template) crée un nouveau ReplicaSet.

Le Deployment de trackr-api

```
apiVersion: apps/v1
kind: Deployment
metadata: {name: trackr-api}
spec:
  replicas: 2
  selector: {matchLabels: {app: trackr-api}}
  template:
    metadata: {labels: {app: trackr-api}}
    spec:
      containers:
        - name: api
          image: ghcr.io/stefanprodan/podinfo:6.9.2
```

- **replicas** : l'état désiré.
- **selector** : les Pods que le Deployment possède ; il doit correspondre aux labels du modèle et ne change plus ensuite.
- **template** : le modèle de Pod ; le modifier déclenche un déploiement.

Mise à l'échelle manuelle

```
kubectl -n trackr scale deploy/trackr-front --replicas=4  
kubectl -n trackr rollout status deploy/trackr-front  
kubectl -n trackr get deploy trackr-front
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
trackr-front	4/4	4	4	5s

- Les deux Pods supplémentaires sont prêts en **moins de 5 secondes** sur le lab.
- Le Service les ajoute tout seul : l'EndpointSlice passe à 4 adresses.
- Attention : un `kubectl apply` du manifeste remet `replicas: 2`.

Le Deployment n'est pas seul



StatefulSet

Nom et volume stables
redis-0, redis-1

Usage type

bases
files de messages



DaemonSet

Un Pod par nœud
même les nouveaux

Usage type

logs
monitoring
CNI



Job

Va jusqu'au succès
completions

Usage type

import
migration



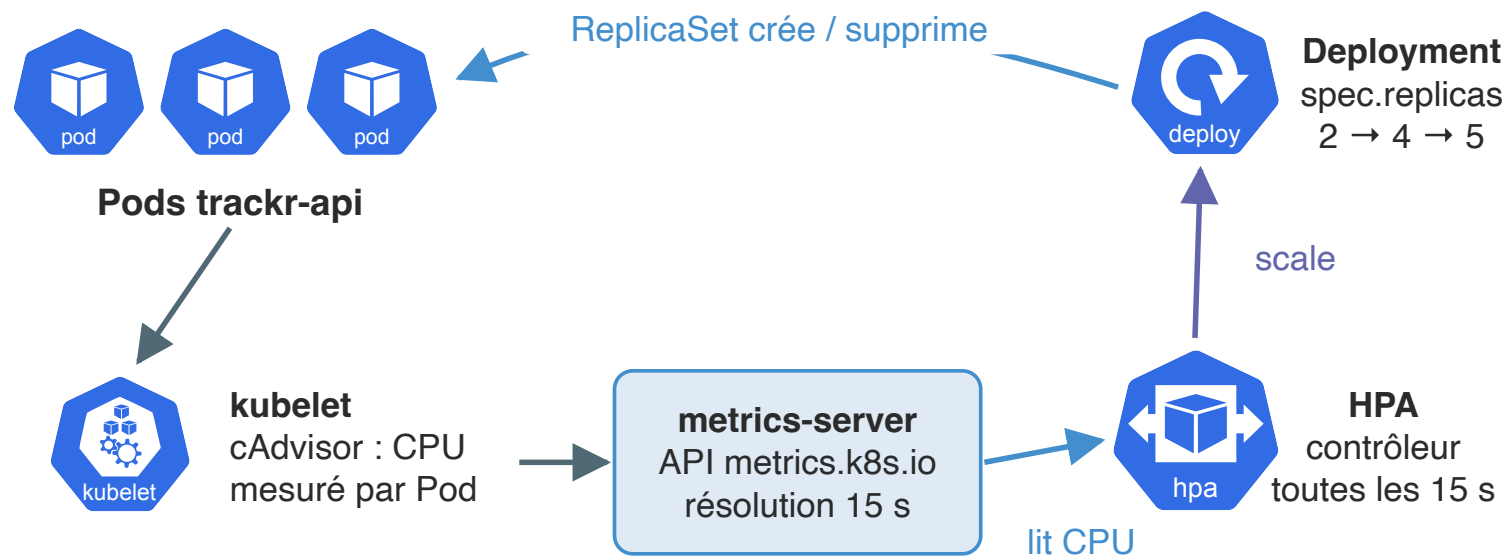
CronJob

Crée un Job planifié
syntaxe cron

Usage type

purge
sauvegarde

HPA : la boucle de régulation



désirés = $\lceil \text{réplicas actuels} \times \text{CPU mesuré} / \text{CPU cible} \rceil$

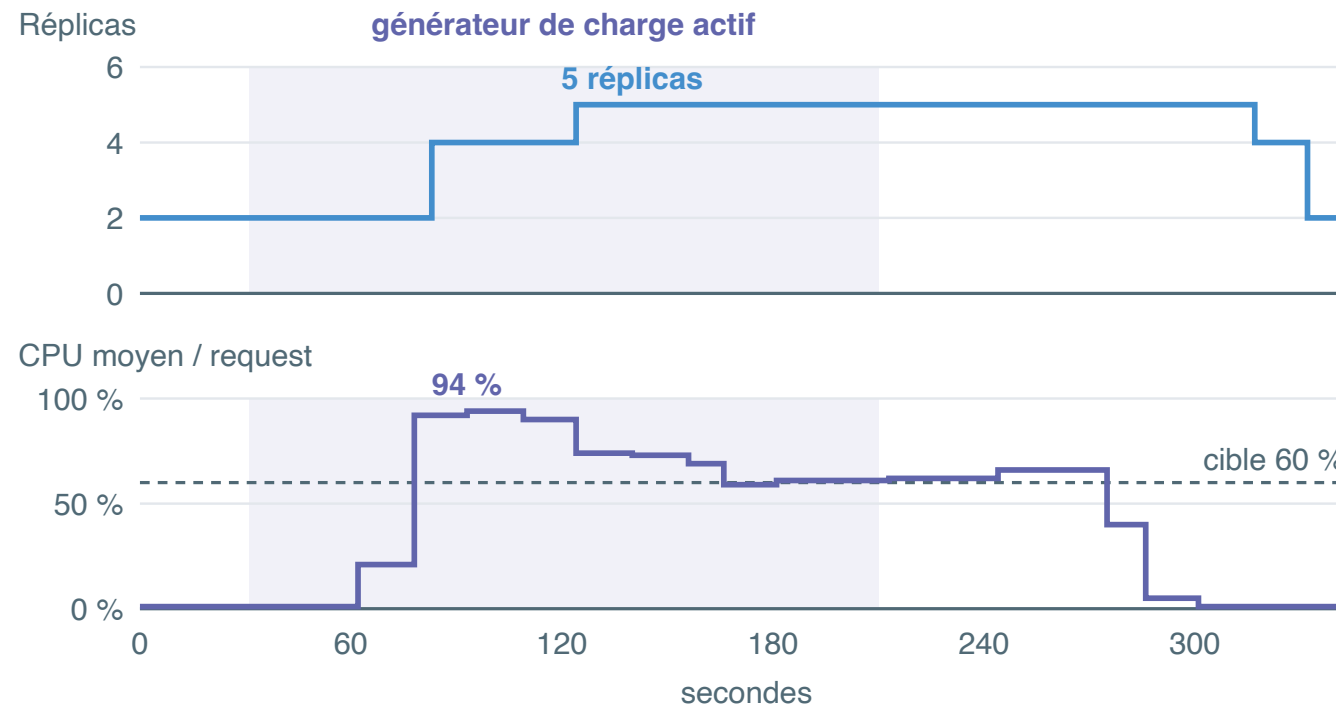
exemple mesuré : $\lceil 2 \times 92 \% / 60 \% \rceil = 4$ réplicas

Déclarer un HPA

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: trackr-api}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: trackr-api}
  minReplicas: 2
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 60}
```

- Cible en **% des requests** : sans `requests.cpu`, le HPA affiche `<unknown>`.
- `behavior.scaleDown` : 300 s de prudence par défaut, 60 s sur TrackR.

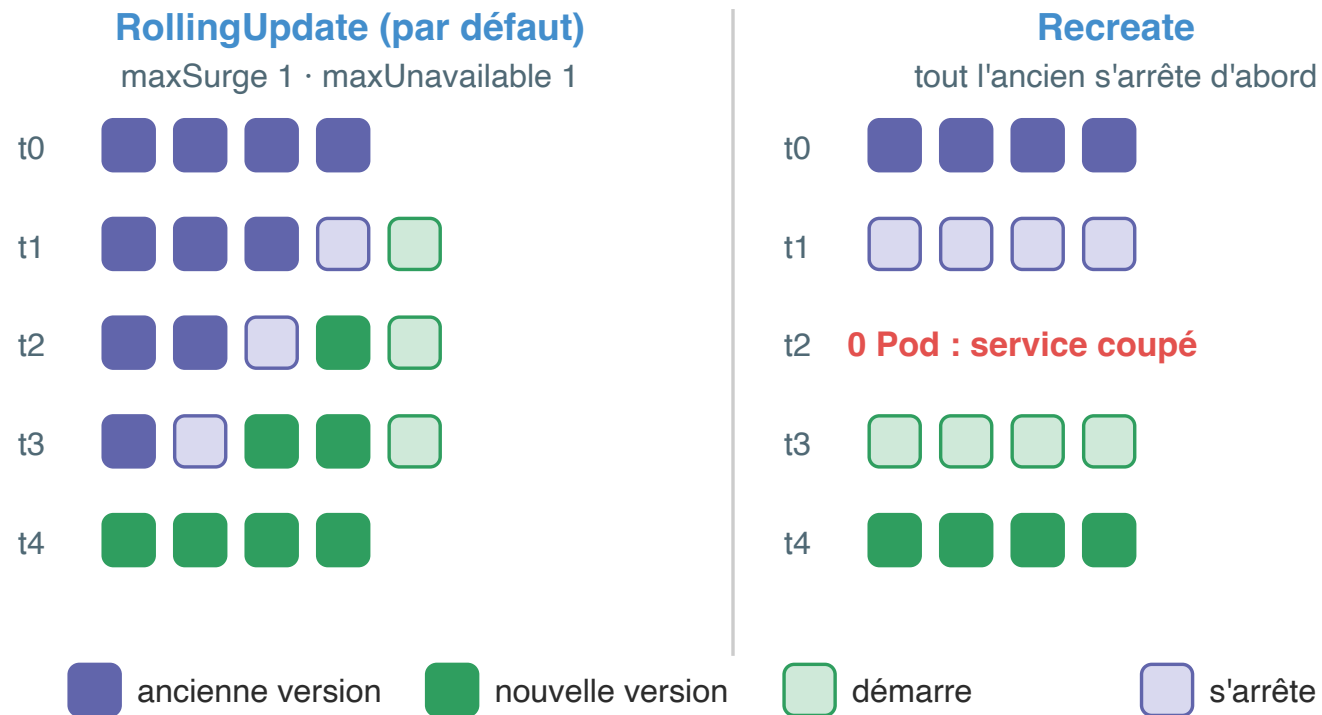
HPA mesuré sur le lab



Charge à +31 s, 4 réplicas à +78 s, 5 à +124 s ; arrêt à +210 s, retour à 2 à +332 s. Mesuré sur le lab, kind v1.35.

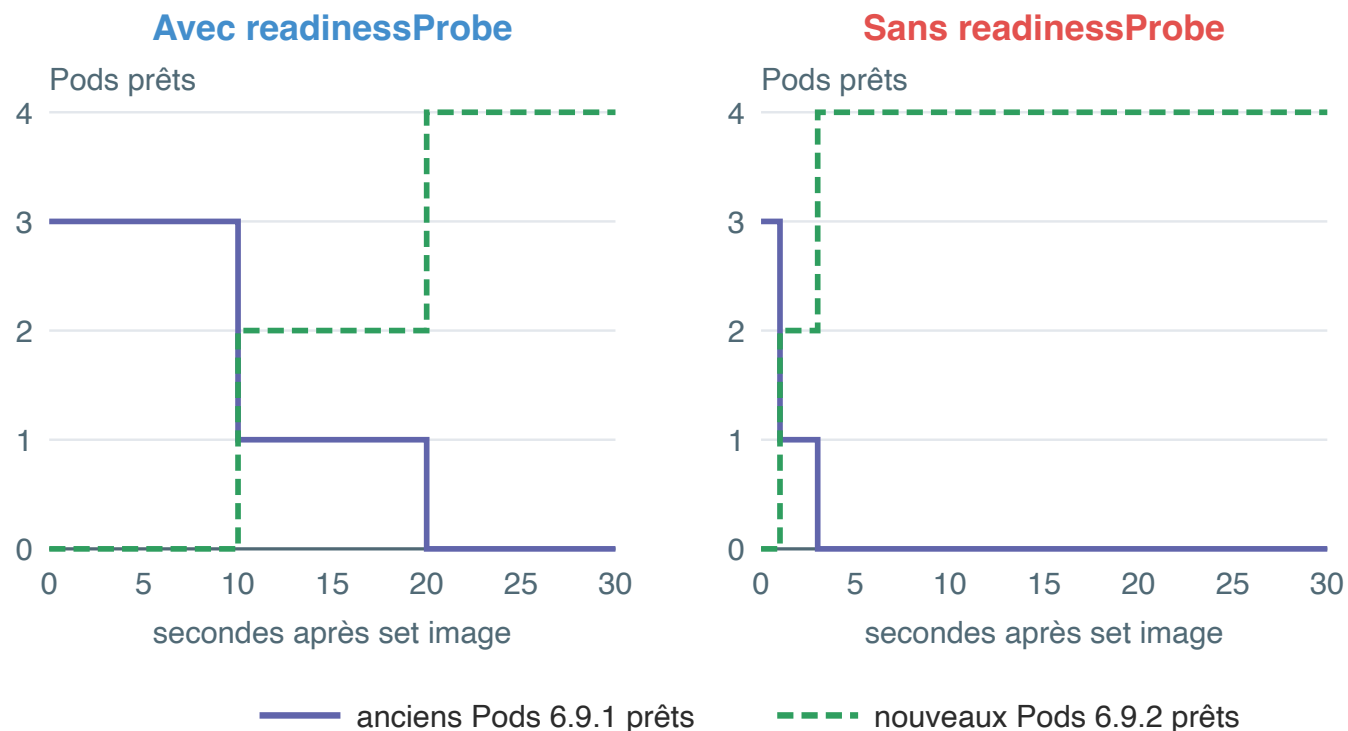
2. Mises à jour et retour arrière

RollingUpdate ou Recreate



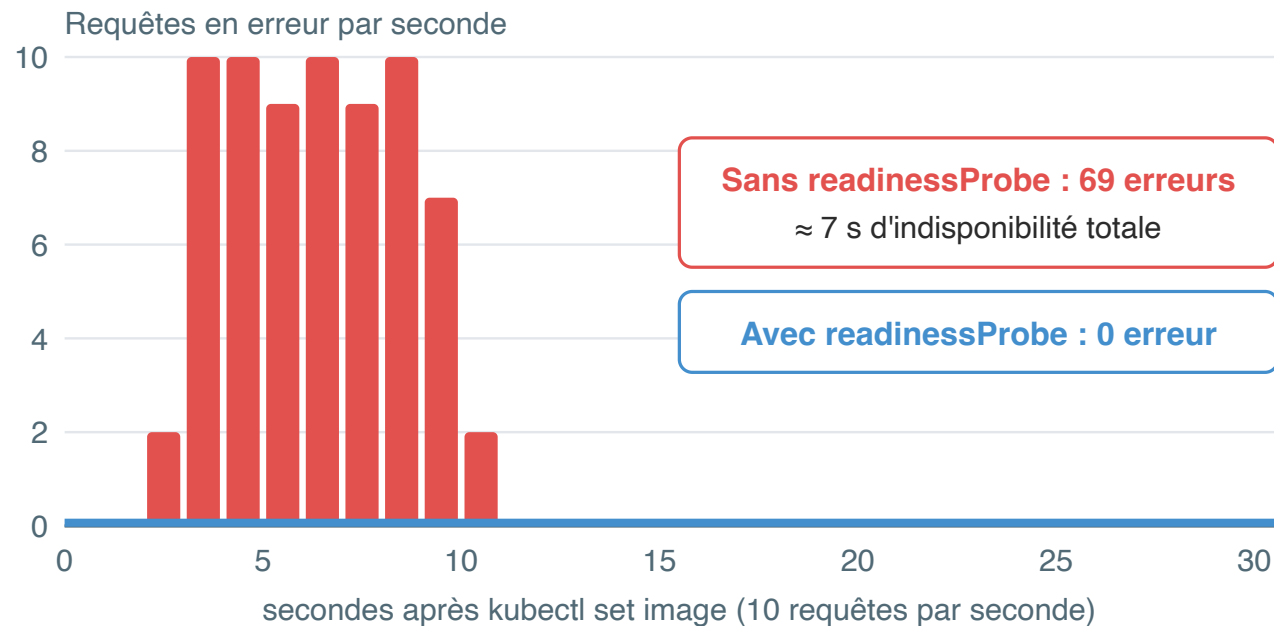
maxSurge : Pods en plus ; **maxUnavailable** : Pods en moins (25 % chacun par défaut).

Rolling update de 4 réplicas, mesuré



6.9.1 vers 6.9.2, démarrage applicatif simulé de 8 s, relevé chaque seconde. Mesuré sur le lab, kind v1.35.

Ce que voient les clients



Sans readinessProbe, un Pod est « prêt » dès que son conteneur tourne, avant d'écouter sur son port.

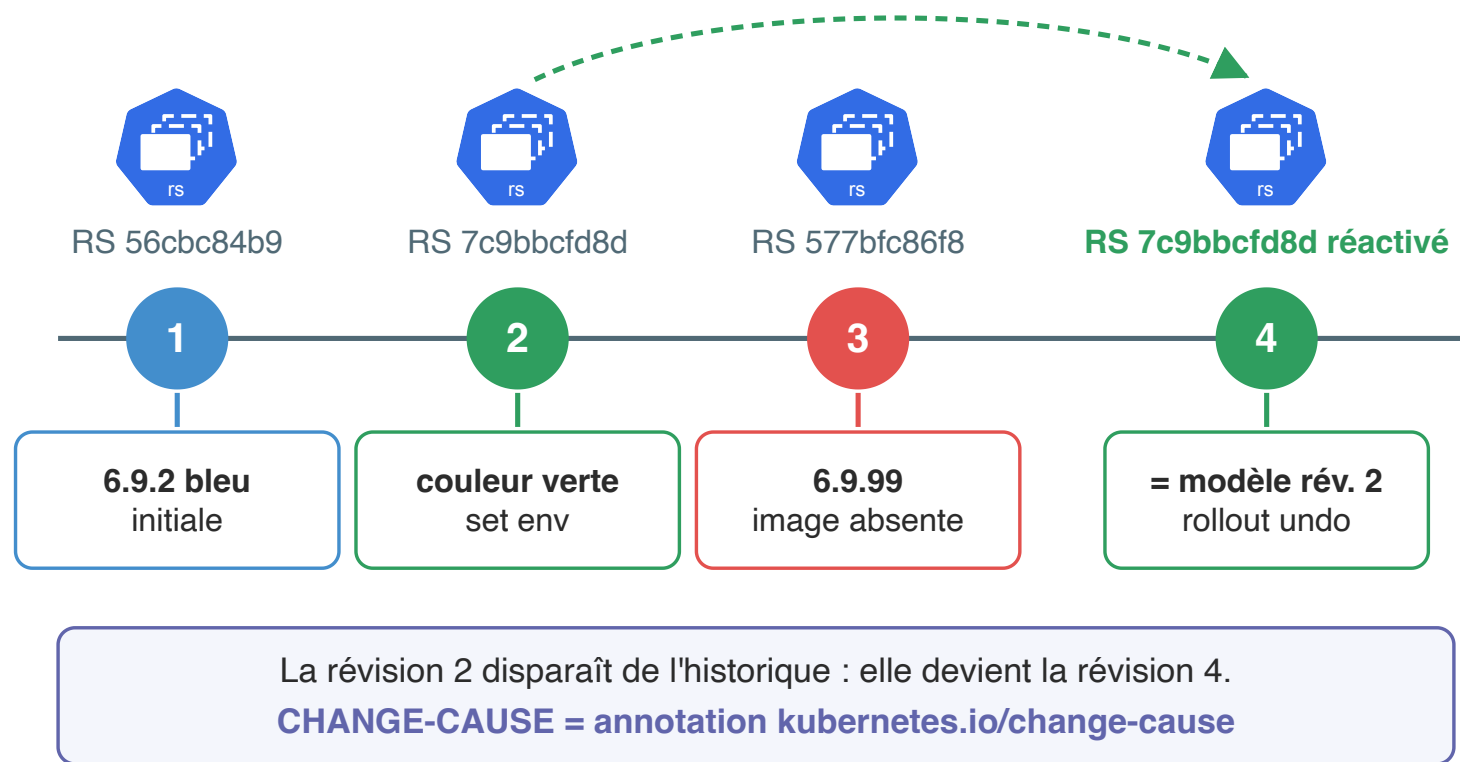
Piloter un rollout

```
kubectl -n trackr set image deploy/trackr-front front=ghcr.io/stefanprodan/podinfo:6.9.99
kubectl -n trackr annotate deploy/trackr-front kubernetes.io/change-cause="Front : 6.9.99"
kubectl -n trackr rollout status deploy/trackr-front --timeout=45s
kubectl -n trackr rollout history deploy/trackr-front
kubectl -n trackr rollout undo deploy/trackr-front # ou --to-revision=1
```

```
error: timed out waiting for the condition
trackr-front-577bfc86f8-z92md 0/1 ImagePullBackOff 0 45s
trackr-front-76b766df7f-t6b6c 1/1 Running 0 78s
```

Avec `maxUnavailable: 0`, les anciens Pods continuent de servir : TrackR reste en ligne.

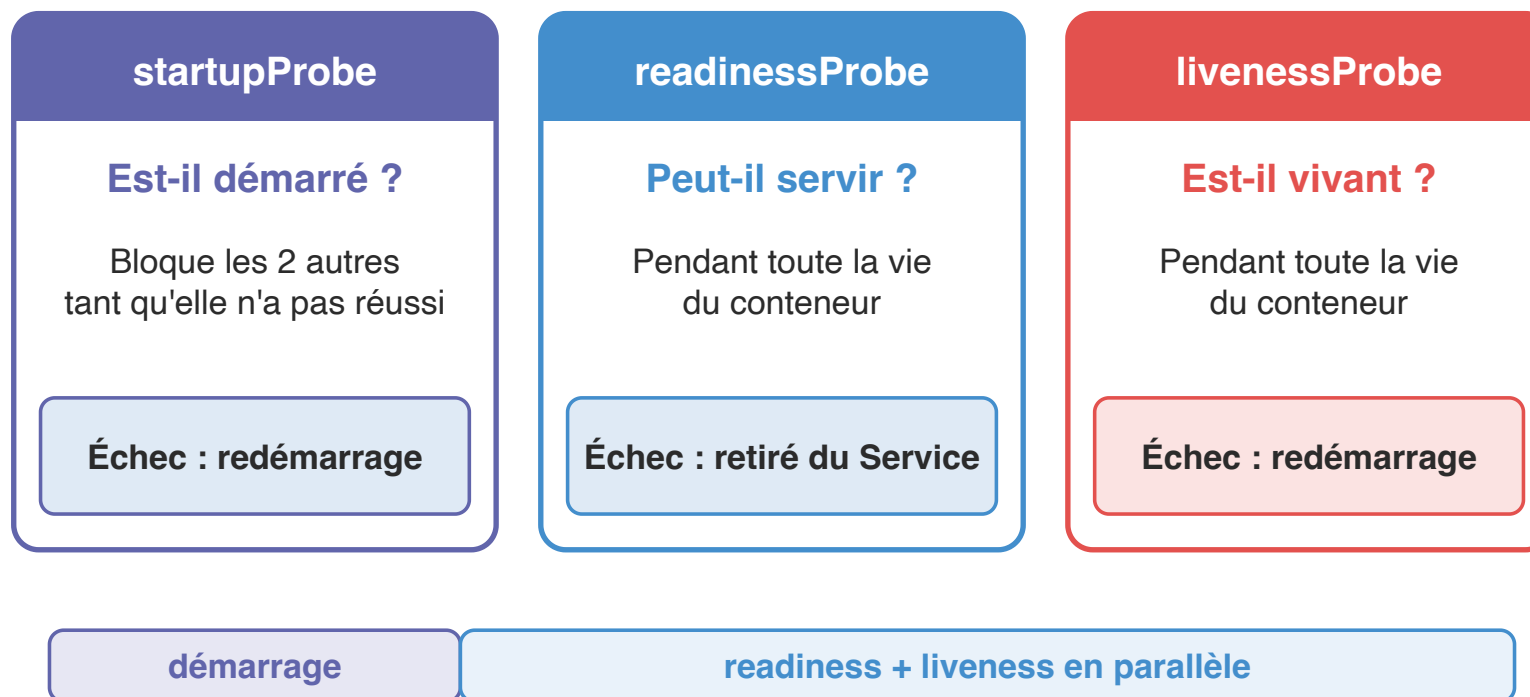
Révisions et change-cause



`rollout undo` avertit désormais : ni le manifeste appliqué ni Git ne sont mis à jour.

3. Sondes de santé

Trois sondes, trois questions



Types de test : httpGet · tcpSocket · exec · grpc

Les sondes de trackr-api

```
startupProbe:
  httpGet: {path: /healthz, port: http}
  periodSeconds: 2
  failureThreshold: 30      # 60 s pour démarrer
readinessProbe:
  httpGet: {path: /readyz, port: http}
  periodSeconds: 3
  failureThreshold: 2
livenessProbe:
  httpGet: {path: /healthz, port: http}
  periodSeconds: 10
```

- **readiness** sur une route qui vérifie la capacité à servir.
- **liveness** sur une route légère, sans dépendance externe : sinon une panne Redis redémarre toutes les API.
- **startup** pour les démarrages lents, au lieu d'un grand `initialDelaySeconds`.

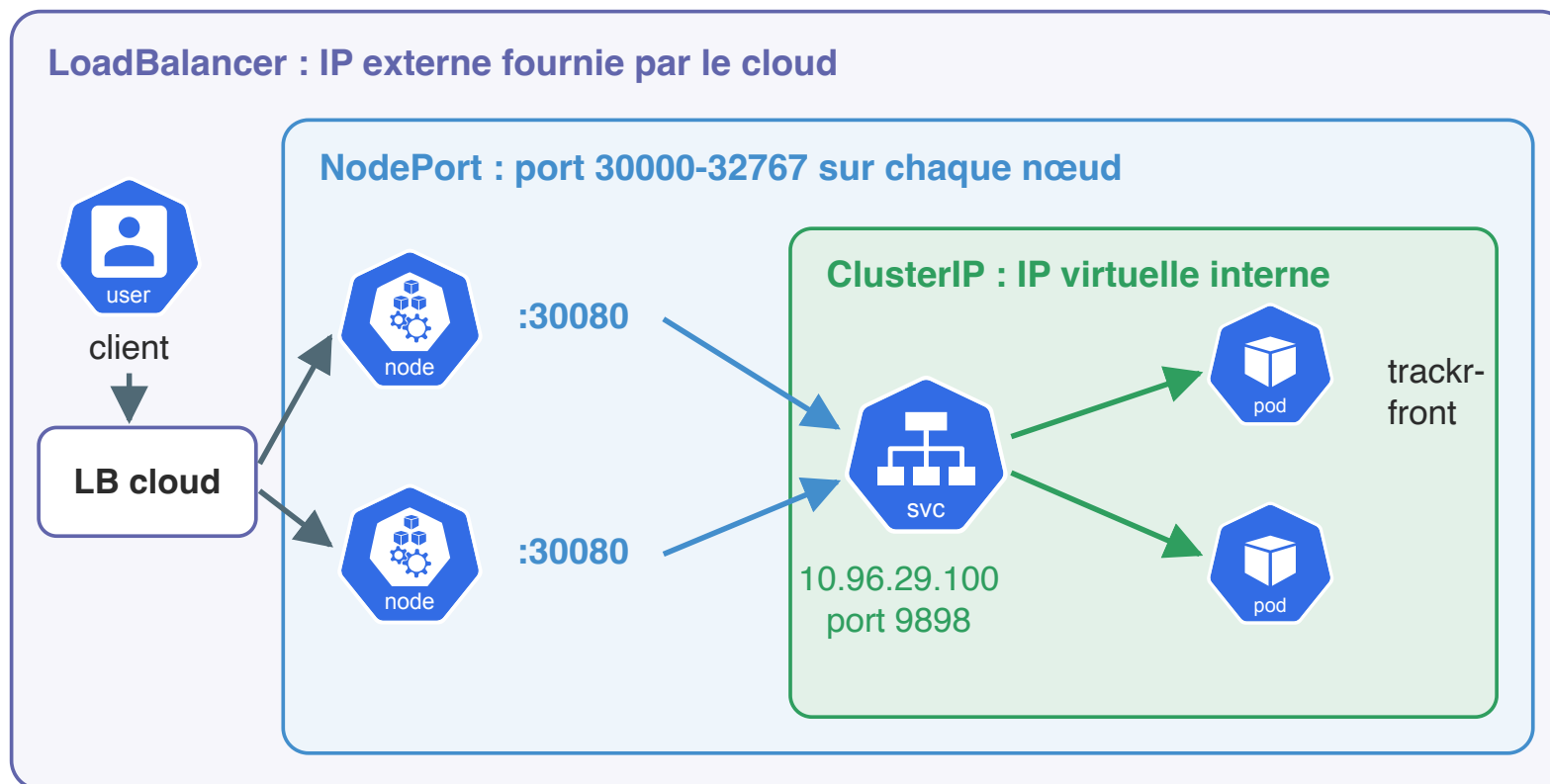
Effet des sondes observé sur le lab

- Liveness en échec (404) : **3 redémarrages en 40 s**, puis des attentes croissantes (`BackOff`).
- Readiness en échec (503) : **0 redémarrage**, mais le Pod est `ready=false` dans l'EndpointSlice et ne reçoit plus de trafic.
- Arrêt d'un Pod : 5 réponses 503 sur 300 pendant deux mises à jour ; avec un `preStop` de 5 s, **0 erreur sur 300**.

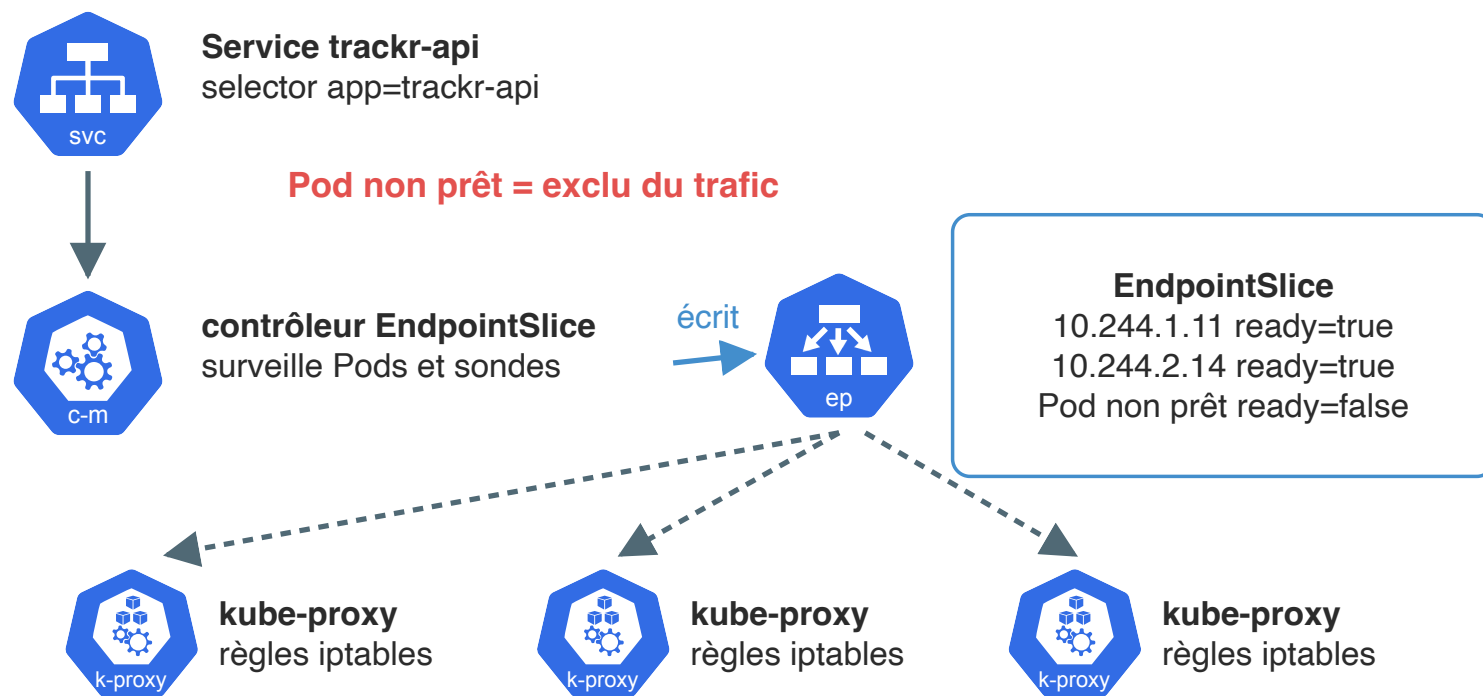


4. Exposer les applications

Les types de Service

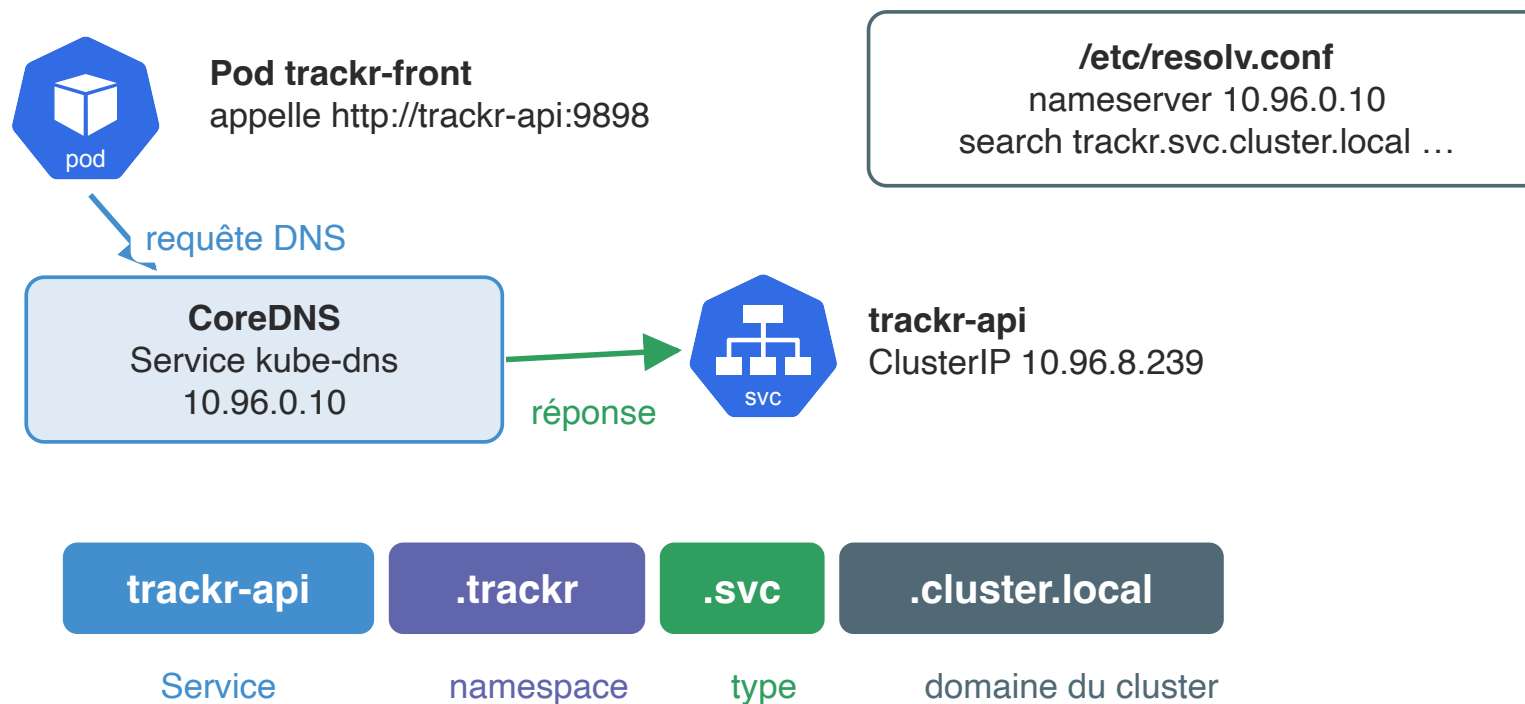


EndpointSlices et kube-proxy



Endpoints (v1) est déprécié depuis la 1.33 : on lit les **EndpointSlices**.

DNS interne



Depuis un autre namespace : **trackr-api.trackr** suffit

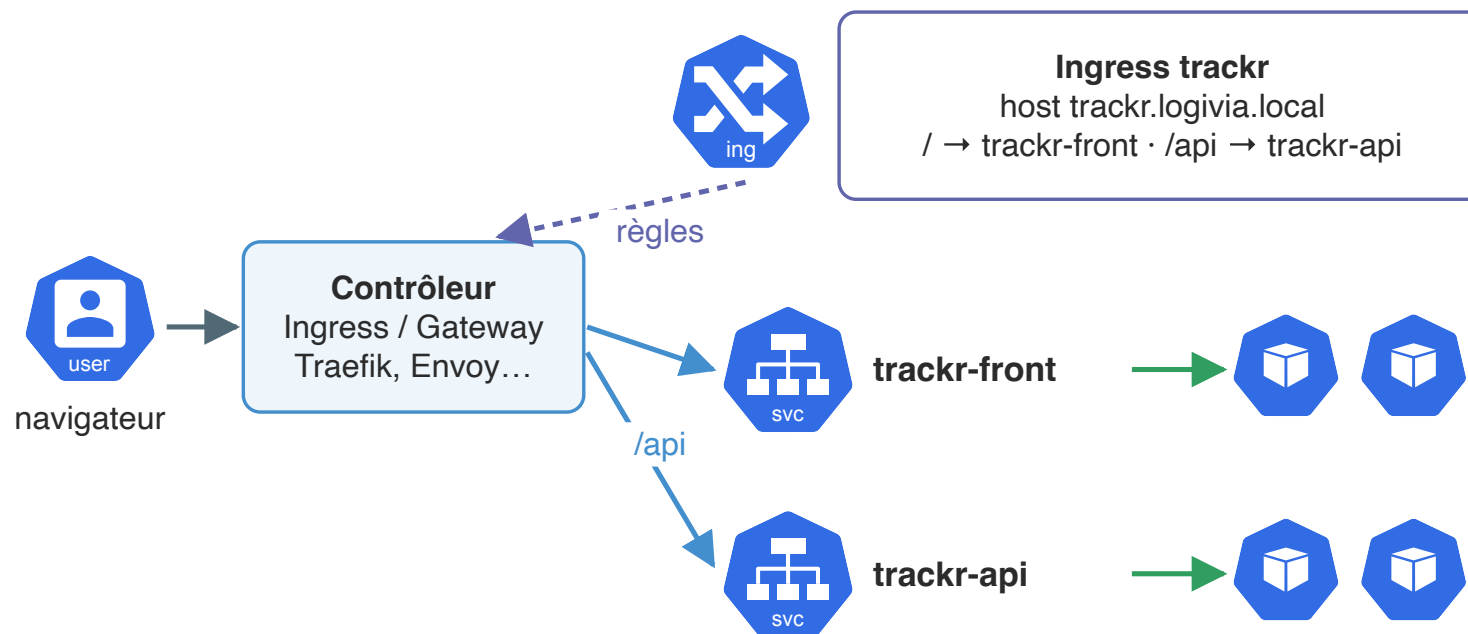
NodePort et LoadBalancer sur le lab

```
$ kubectl -n trackr get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
trackr-api	ClusterIP	10.96.8.239	<none>	9898/TCP
trackr-api-lb	LoadBalancer	10.96.241.100	<pending>	80:32273/TCP
trackr-api-nodeport	NodePort	10.96.119.189	<none>	9898:31493/TCP
trackr-front	NodePort	10.96.29.100	<none>	9898:30080/TCP

- `http://localhost:30080` : le port 30080 du control-plane kind est publié sur l'hôte.
- `<pending>` : sans contrôleur cloud, aucune IP externe n'est attribuée (en cloud : un équilibreur est créé).

Ingress et Gateway API



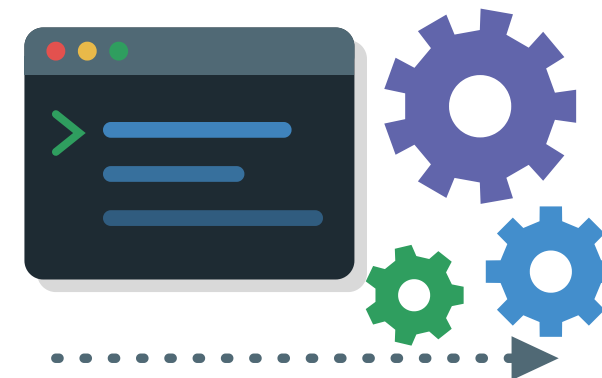
Une entrée HTTP(S) unique, routage par hôte et chemin, TLS terminé à l'entrée

ingress-nginx est retiré (mars 2026) : préférer la Gateway API. Non joué sur le lab.

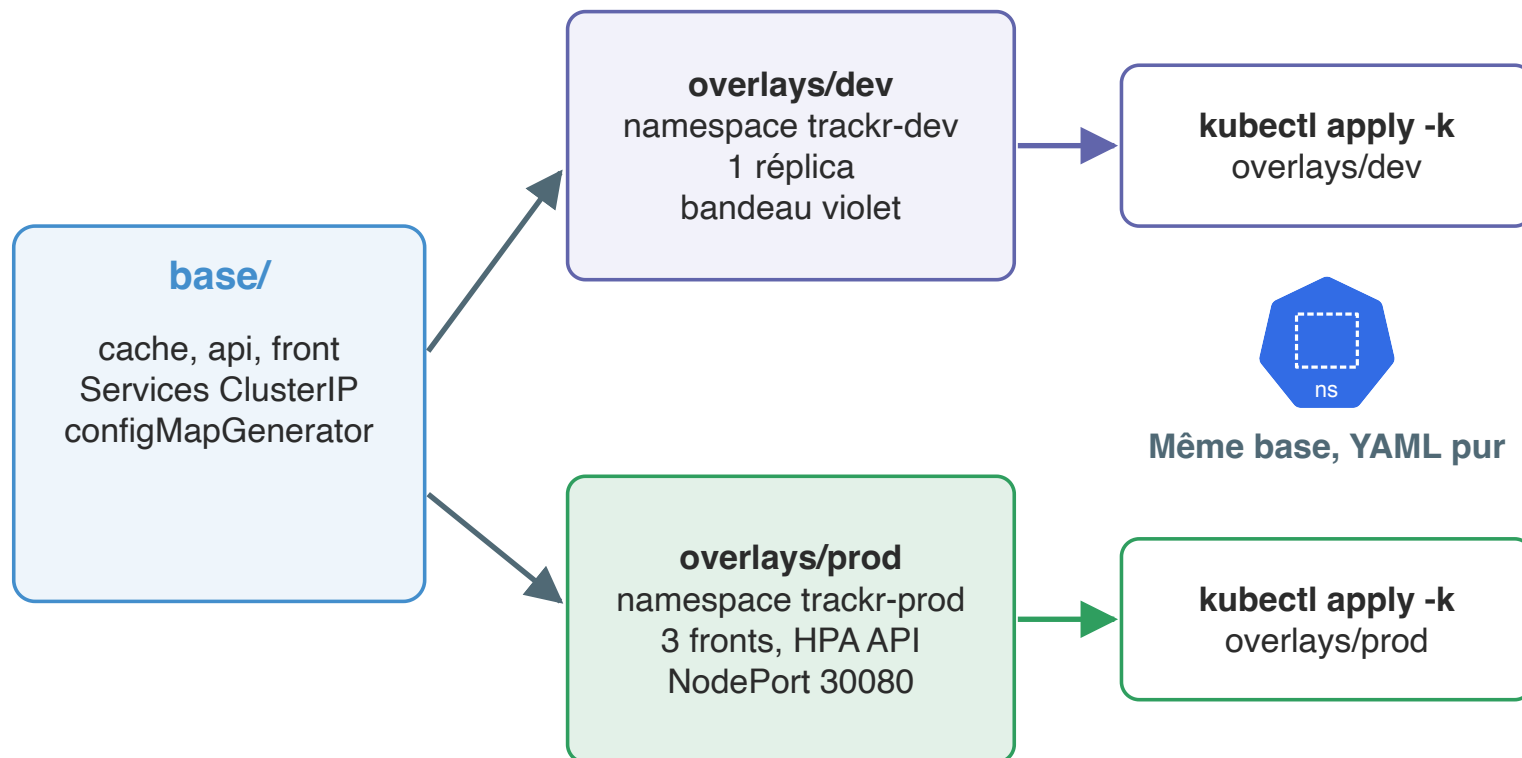
5. Automatiser les déploiements

Déclaratif d'abord

- Les manifestes vivent dans **Git** : ils sont la source de vérité.
- `kubectl apply -f` crée ou met à jour ;
`kubectl diff -f` montre l'écart avant d'appliquer.
- Les commandes impératives (`scale`, `set image`, `rollout undo`) servent au dépannage, puis on reporte la modification dans Git.



Kustomize : une base, des overlays



Kustomize et Helm en pratique

```
kubectl kustomize overlays/dev # rendu  
kubectl diff -k overlays/dev  
kubectl apply -k overlays/dev
```

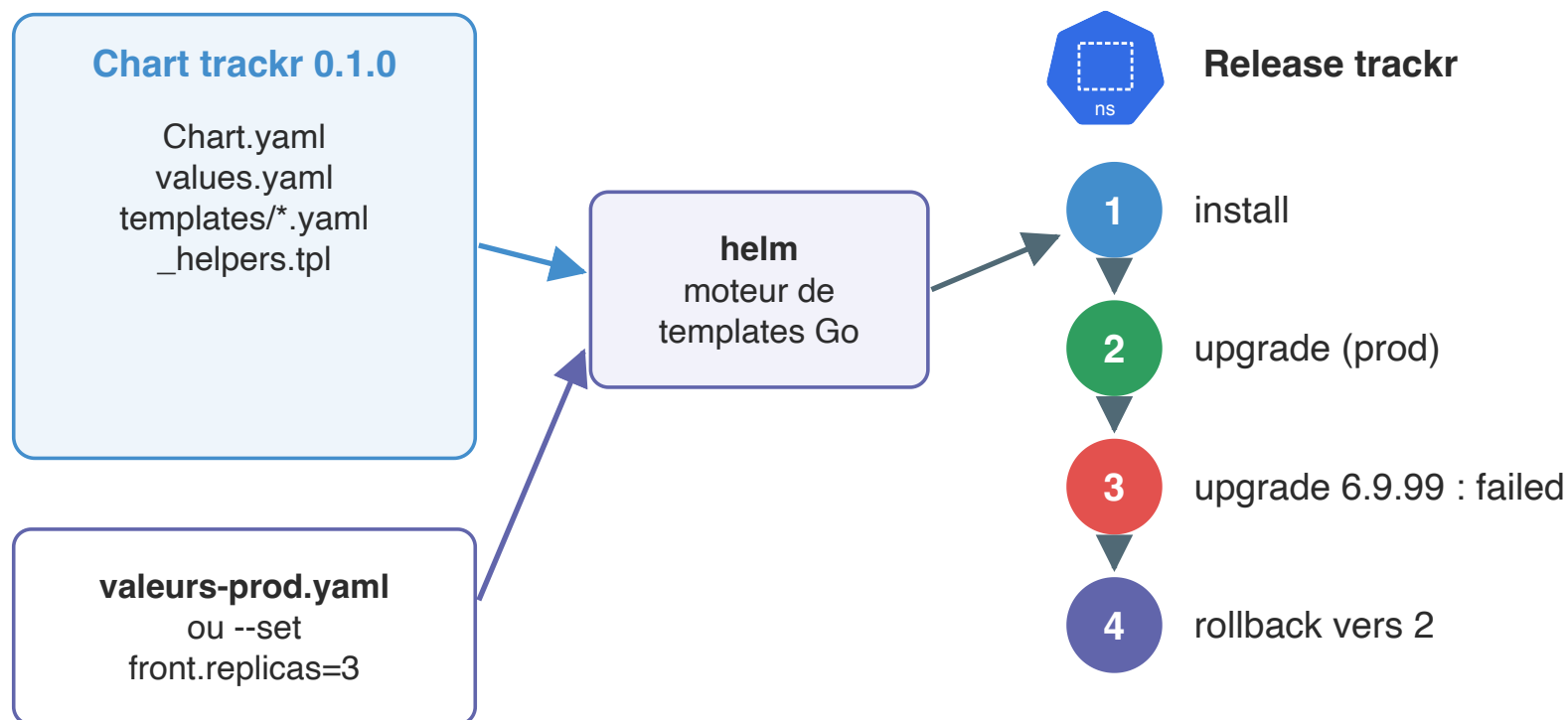
```
configmap/trackr-front-config-h5c95775m2 created  
deployment.apps/trackr-front configured
```

Modifier la ConfigMap générée change son suffixe : le front redémarre seul.

```
helm install trackr ./trackr -n trackr  
helm upgrade trackr ./trackr -n trackr \\\n  -f trackr/valeurs-prod.yaml  
helm history trackr -n trackr  
helm rollback trackr 2 -n trackr
```

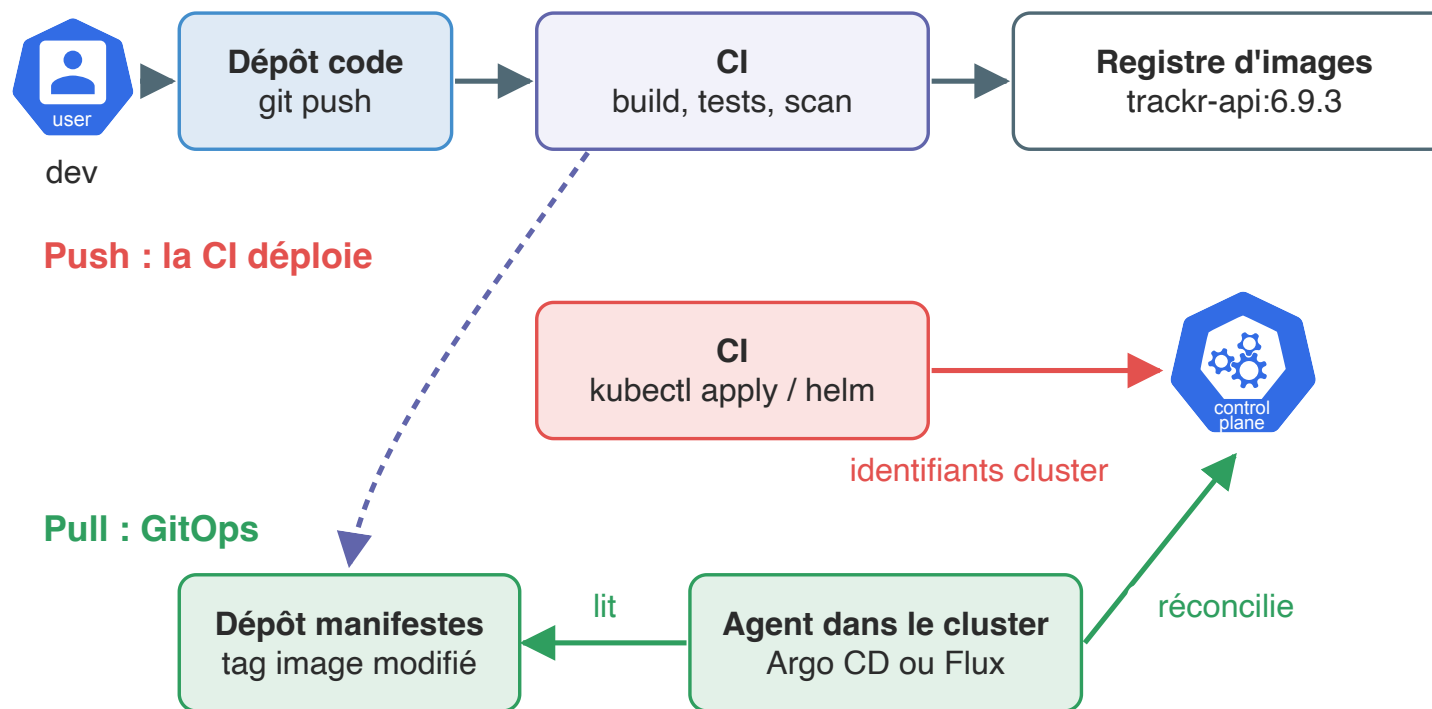
```
3  failed    Upgrade "trackr" failed  
4  deployed  Rollback to 2
```

Helm : chart, valeurs et releases



Historique stocké dans des Secrets sh.helm.release.v1.trackr.vN

CI/CD et GitOps



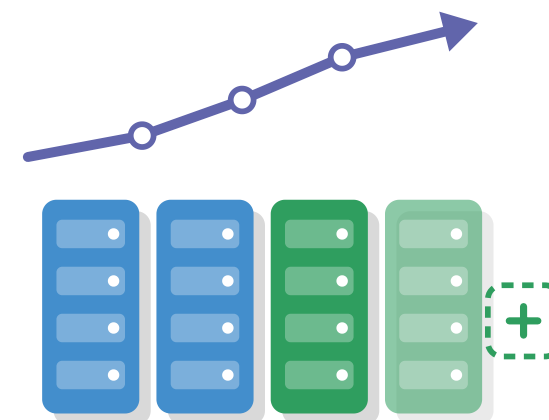
En GitOps, aucun identifiant du cluster ne sort du cluster. Non joué sur le lab.

Démos et TP du module

Démo 5 — `demos/demo-05-rolling-update-hpa.md` : rolling update avec et sans readinessProbe, rollback, HPA sous charge.

Démo 5b — `demos/demo-05b-kustomize-helm.md` : overlays dev et prod, chart Helm, upgrade et rollback.

TP 5 — `tp/tp-05-deployer-trackr.md` : déployer TrackR complet, mise à jour sans interruption, rollback, HPA.



À retenir

- Deployment → ReplicaSet → Pods : on déclare l'état, les contrôleurs le maintiennent.
- `maxUnavailable: 0` + readinessProbe + preStop : **0 erreur** mesurée pendant les mises à jour.
- Readiness retire du trafic, liveness redémarre : ne pas les confondre.
- Un Service suit les Pods prêts via les EndpointSlices ; le DNS donne `svc.namespace.svc.cluster.local`.
- Kustomize adapte du YAML, Helm le génère et versionne les releases ; Git reste la source de vérité.